

ORB

OmniReconnaissance Bot

YOLO-based planforhuman+weapons(knife,gun,otherhandheldweapons)detection on a Raspberry Pi 4

High-level architecture (what you'll build)

- Two cameras → capture frames (synchronized-ish) → preprocessor → YOLO detector (person, gun, knife, other weapons) → postprocess (NMS, confidence threshold) → overlay + stream to Flask/WebRTC dashboard + alert system (MQTT / buzzer).

Software stack & libraries

- Raspberry Pi OS (64-bit) or Ubuntu 22.04 64-bit (recommended for ONNX/OpenVINO support).
- Python 3.9+ virtualenv
- OpenCV (with libcamera / v4l2 support)
- Ultralytics ultralytics (for training/export) — training on desktop/GPU, not Pi. [Ultralytics Docs](#)
- ONNX Runtime and/or OpenVINO for acceleration on Pi. ONNX Runtime has Pi guides; OpenVINO provides optimization flow for YOLOv8

Dual-camera inference program (Python)

- handles two camera streams (CSI via libcamera or v4l2 and USB via /dev/video*),
- runs ONNX Runtime inference (single-threaded or multithreaded),
- does preprocessing and postprocessing (NMS),
- draws boxes and sends frames to a Flask MJPEG endpoint.

Performance tuning & optimization tips

- Use yolov8n (nano) for edge. If speed allows, try yolov8s.
- Export to ONNX then run with OpenVINO or ONNXRuntime on Pi for better perf. OpenVINO has tutorials for optimizing YOLOv8. [OpenVINO Documentation+1](#)
- Quantize model (FP16 / INT8) if supported by conversion tool — reduces CPU ops.
- Use frame-skipping: run inference on every nth frame (e.g., process at 5–8 FPS, capture at 15–30 FPS).
- Use threading/process pools for capture + inference to keep cameras streaming while inference runs.
- Reduce input imgsz to 320–640 depending on tradeoff.

Raspberry Pi Setup

2.2 Boot Pi & Update

Insert the SD Card → Power ON → Open Terminal

```
sql

sudo apt update && sudo apt upgrade -y
```

 Copy code

Enable cameras and interfaces:

```
arduino

sudo raspi-config
```

 Copy code

Enable following:

- ✓ Camera (CSI)
- ✓ I2C (thermal sensor)
- ✓ SPI (optional)
- ✓ SSH
- ✓ VNC (optional)

Reboot:

```
nginx

sudo reboot
```

 Copy code



3.1 Install Python 3 modules

```
rust

sudo apt install python3-pip python3-opencv -y
pip install numpy flask flask-socketio eventlet
pip install ultralytics onnxruntime
pip install imutils
pip install adafruit-circuitpython-mlx90640
```

 Copy code

3.2 Install Libcamera (for CSI Camera)

Already included, but verify:

```
libcamera-still -o test.jpg
```

 Copy code

3.3 Install GStreamer (for streaming feeds to dashboard)

csharp

 Copy code

```
sudo apt install gstreamer1.0-tools gstreamer1.0-plugins-base \  
gstreamer1.0-plugins-good gstreamer1.0-plugins-bad \  
gstreamer1.0-plugins-ugly -y
```

RC-WL0516 Microwave Radar Motion Sensor to a Raspberry Pi 4.

✓ 1. What the RC-WL0516 Sensor Pins Mean

The RC-WL0516 has 3 pins:

Pin	Meaning	Connect to Pi
VCC	Power +5V	5V pin (Pin 2 or 4)
OUT	Digital output (HIGH when motion detected)	GPIO pin (e.g., GPIO17, Pin 11)
GND	Ground	GND pin (Pin 6, 9, 14, etc.)

⚠ Output is 3.3V logic, safe for Raspberry Pi.

✓ 2. Raspberry Pi Pinout (Beginner Safe)

Use this map:

SCSS Copy code

(Top view)

3V3 (1) (2) 5V

GPIO2(3) (4) 5V

GPIO3(5) (6) GND

GPIO4(7) (8) GPIO14

GND (9) (10) GPIO15

GPIO17(11)(12) GPIO18

...

We will use:

- 5V → power sensor
- GND → ground
- GPIO17 (Pin 11) → read motion signal



✓ 3. Wiring Diagram (Very Simple)

sql

 Copy code

```
RC-WL0516 ----- Raspberry Pi 4
-----
VCC -----> 5V (Pin 2 or 4)
GND -----> GND (Pin 6)
OUT -----> GPIO17 (Pin 11)
```

ASCII Diagram:

sql

 Copy code

```
RC-WL0516
-----
| VCC   (5V)   |-----> Pi Pin 2 (5V)
| OUT   (Dout) |-----> Pi Pin 11 (GPIO17)
| GND                |-----> Pi Pin 6 (GND)
-----
```

✓ 4. Enable GPIO Library

Install:

bash

 Copy code

```
sudo apt install python3-gpiozero
```

✓ 5. Python Code to Read Radar Sensor

Create a file:

bash

 Copy code

```
nano radar.py
```

Paste:

python

 Copy code

```
from gpiozero import InputDevice
import time

radar = InputDevice(17)  # OUT pin → GPIO17

print("RC-WL0516 Radar Ready...")

while True:
    if radar.is_active:
        print("MOTION DETECTED")
    else:
        print("No motion")

    time.sleep(0.5)
```

Run:

bash

 Copy code

```
python3 radar.py
```



STEP 3 — Fusion Logic (Actual Behavior)

sql

Radar = LOW → No motion → YOLO OFF → show idle screen

Radar = HIGH → Motion detected:

- YOLO activates
- Capture frames from both cameras
- Run inference
- Detect Person / Gun / Knife
- Alert + Display live stream

Full Fusion Code

Create file:

bash

```
nano radar_yolo_fusion.py
```

AMG8833 (8×8 thermal sensor) with Raspberry Pi 4,

What is the AMG8833 (Adafruit Grid-Eye 8×8)?

64-pixel thermal camera

Detects temperatures: 0°C to ~80°C

Distance: best at 0.5m – 2m

Communicates via I2C → perfect for Raspberry Pi 4

Produces a thermal heatmap you can overlay on YOLO video feed

✓ 2. AMG8833 → Raspberry Pi 4 Wiring (I2C)

AMG8833 has 4 pins:

AMG8833 Pin	Connect to Pi 4
VIN / VCC	5V (Pin 2 or 4)
GND	GND (Pin 6 or 14)
SDA	SDA (GPIO 2 – Pin 3)
SCL	SCL (GPIO 3 – Pin 5)

✓ 3. Enable I2C on Raspberry Pi

Run:

```
bash
```

[Copy code](#)

```
sudo raspi-config
```

Go to:

```
pgsql
```

[Copy code](#)

```
Interface Options → I2C → Enable
```

Install I2C tools:

```
bash
```

[Copy code](#)

```
sudo apt update
```

```
sudo apt install -y i2c-tools python3-pip
```

Reboot:

```
bash

sudo reboot
```

 Copy code

Check if sensor is detected:

```
bash

i2cdetect -y 1
```

 Copy code

You should see address 0x69.

4. Install Required Python Libraries

```
bash

pip3 install adafruit-circuitpython-amg88xx numpy scipy pillow opencv-python
```

 Copy code

Enable I2C Python support:

```
bash

sudo pip3 install smbus2
```

 Copy code

5. Basic Test Code – Read Temperature Grid

Create file:

```
bash

nano amg8833_test.py
```

 Copy code

Paste:

```
python

import time
import board
import busio
import adafruit_amg88xx

i2c = busio.I2C(board.SCL, board.SDA)
sensor = adafruit_amg88xx.AMG88XX(i2c)

while True:
    pixels = sensor.pixels
    for row in pixels:
        print("{0:.1f}".format(temp) for temp in row)
    print("\n---- FRAME ----\n")
    time.sleep(0.5)
```

 Copy code

Run:

```
bash

python3 amg8833_test.py
```

 Copy code

You should see 8×8 temperature values updating in real-time.

✓ 6. Generate Thermal Heatmap (Color Image)

Create file:

bash

Copy code

nano amg_heatmap.py

Paste:

python

Copy code

```
import time
import numpy as np
import cv2
import board
import busio
import adafruit_amg88xx

i2c = busio.I2C(board.SCL, board.SDA)
sensor = adafruit_amg88xx.AMG88XX(i2c)

while True:
    # Read 8x8 thermal grid
    pixels = np.array(sensor.pixels)

    # Normalize temperature to 0-255
    normalized = cv2.normalize(pixels, None, 0, 255, cv2.NORM_MINMAX)
    normalized = np.uint8(normalized)
```

```
# Resize to larger heatmap
heatmap = cv2.resize(normalized, (480, 480), interpolation=cv2.INTER_CUBIC)

# Apply color map
heatmap_color = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)

cv2.imshow("AMG8833 Heatmap", heatmap_color)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cv2.destroyAllWindows()
```

Run:

bash

Copy code

```
python3 amg_heatmap.py
```

HB-100 microwave sensor to a Raspberry Pi 4

HB-100 Pinout

The HB-100 usually has three main pins:

Vcc (5V) – Power supply

GND – Ground

IF / OUT – Analog signal output (motion signal)

Some HB-100 boards also have a LO (local oscillator) and optional antenna connectors, but the main focus is Vcc, GND, and IF.

Powering HB-100

Connect Vcc → 5V pin on Pi 4 (Pin 2 or Pin 4)

Connect GND → GND on Pi 4 (Pin 6, 9, 14, etc.)

Ensure your Pi power supply can handle additional current (~40-50 mA per HB-100).

Signal Connection

The IF output of HB-100 is an analog signal (~100–200 mV amplitude AC). Since Pi 4 does not have ADC, you'll need:

Option A – ADC Module

Use an ADC like MCP3008 to read the analog output.

IF → ADC channel input

MCP3008 communicates with Pi via SPI

Read the analog value in Python

Option B – Signal Conditioning for GPIO

Use a comparator or op-amp to convert the AC output into digital pulses

Output can then go to a Pi GPIO pin for motion detection

4. Example Wiring (with MCP3008)

mathematica

```
HB-100 Vcc → Pi 5V
HB-100 GND → Pi GND
HB-100 IF → MCP3008 CH0
MCP3008 VDD/VREF → Pi 3.3V
MCP3008 GND → Pi GND
MCP3008 CLK → Pi SCLK (GPIO11)
MCP3008 DOUT → Pi MISO (GPIO9)
MCP3008 DIN → Pi MOSI (GPIO10)
MCP3008 CS → Pi CE0 (GPIO8)
```

MLX90640 thermal camera to a Raspberry Pi 4

Hardware Connection

The MLX90640 communicates via I²C, so you'll connect it to the Pi's I²C pins.

MLX90640 Pins:

VDD → 3.3V (Pin 1 on Pi)

GND → Ground (Pin 6 on Pi)

SDA → I²C Data (Pin 3 on Pi, GPIO2)

SCL → I²C Clock (Pin 5 on Pi, GPIO3)

AD0 → Connect to GND or leave floating (sets I²C address, default 0x33)

INT → Optional, for interrupts (can be left unconnected initially)

2. Enable I²C on the Raspberry Pi

1. Open terminal:

```
bash
```

[Copy code](#)

```
sudo raspi-config
```

2. Go to **Interface Options** → **I²C** → **Enable**.

3. Install I²C tools:

```
bash
```

[Copy code](#)

```
sudo apt update
```

```
sudo apt install -y i2c-tools python3-smbus
```

4. Check if sensor is detected:

```
bash
```

[Copy code](#)

```
i2cdetect -y 1
```

You should see `0x33` (default MLX90640 address).

3. Install Python Libraries

Install the official Adafruit MLX90640 library:

```
bash
```

```
pip3 install adafruit-circuitpython-mlx90640
```

```
pip3 install numpy matplotlib
```

4. Test the Sensor

Here's a basic Python script to read temperature frames:

python

 Copy code

```
import time
import board
import busio
import adafruit_mlx90640
import numpy as np
import matplotlib.pyplot as plt

# Setup I2C
i2c = busio.I2C(board.SCL, board.SDA, frequency=400000)
mlx = adafruit_mlx90640.MLX90640(i2c)
mlx.refresh_rate = adafruit_mlx90640.RefreshRate.REFRESH_2_HZ # 2 Hz
```

```
frame = np.zeros((24*32,))

plt.ion()
fig, ax = plt.subplots()
im = ax.imshow(frame.reshape((24,32)), cmap='inferno', vmin=20, vmax=40)
plt.colorbar(im)

while True:
    try:
        mlx.getFrame(frame)
        im.set_data(frame.reshape((24,32)))
        plt.draw()
        plt.pause(0.01)
    except KeyboardInterrupt:
        break
```

Explanation:

The sensor outputs a 24x32 thermal matrix.

RefreshRate can be set from 0.5Hz to 16Hz.

You can plot a heatmap using Matplotlib for your dashboard.